

TRUY VẤN DỮ LIỆU

Bài 5. Truy vấn dựa trên ngữ nghĩa

PGS.TS. Lê Thanh Hương
Trưởng Công nghệ thông tin và Truyền thông
Đại học Bách Khoa Hà Nội

- Các vấn đề đối với tìm kiếm dựa trên so khớp từ
- Word vectors
- Transformer
- BERT
- Sentence-BERT
- Sentence Transformer
- Bi-encoder
- Inverted index file
- ColBERT
- Condenser
- Datasets

Các vấn đề đối với tìm kiếm dựa trên so khớp từ

- Xử lý từ phức, từ ghép, từ biến đổi hình thái
 - I love this move -> I loved this movie

Subword
Models

- Xử lý từ đồng nghĩa
 - I love this move -> I adore this movie

Word
Embeddings

- Xử lý kết hợp đặc trưng
 - I love this movie -> I don't love this movie
 - I hate this movie -> I don't hate this movie

Neural
Networks

- Xử lý cấu trúc câu
 - It has an interesting story, but is boring overall

Sequence
Models

Các vấn đề đối với tìm kiếm dựa trên so khớp từ

Query: Where was the last world cup?

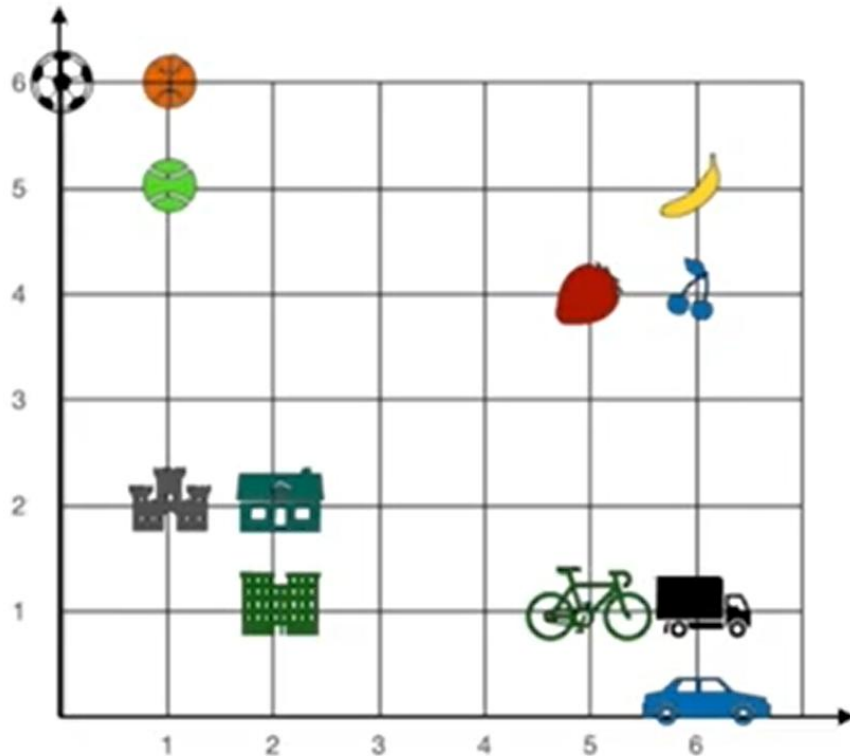
Sentences	
The last world cup was in Qatar	5
The sky is blue	1
The bear lives in the woods	2
An apple is a fruit	0

Các vấn đề đối với tìm kiếm dựa trên so khớp từ

Query: Where was the last world cup?

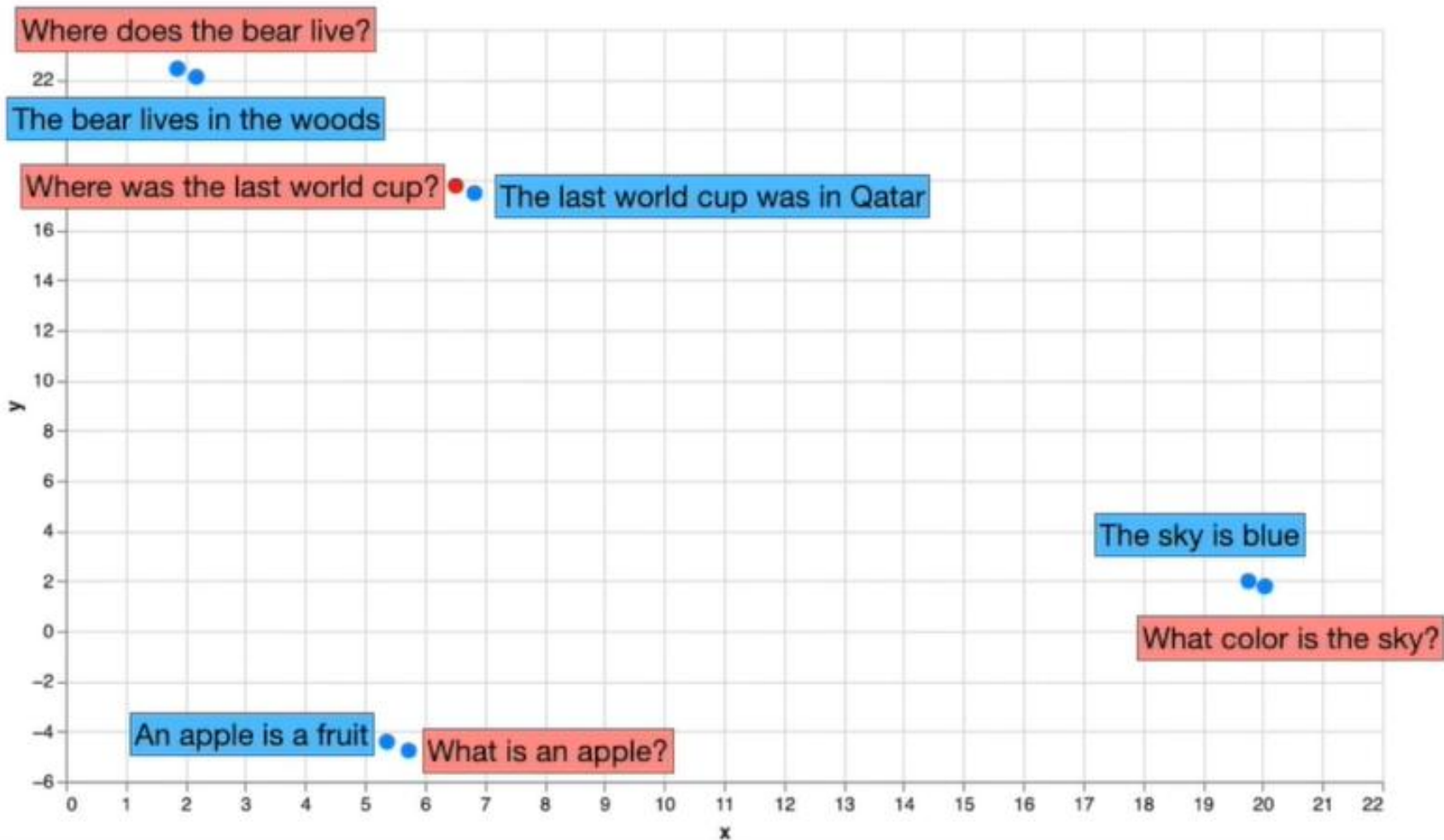
Sentences	
The previous world cup was in Qatar	4
The cup is where you left it	3
Where in the world is my last cup of coffee?	5
An apple is a fruit	0

Biểu diễn vec tơ từ

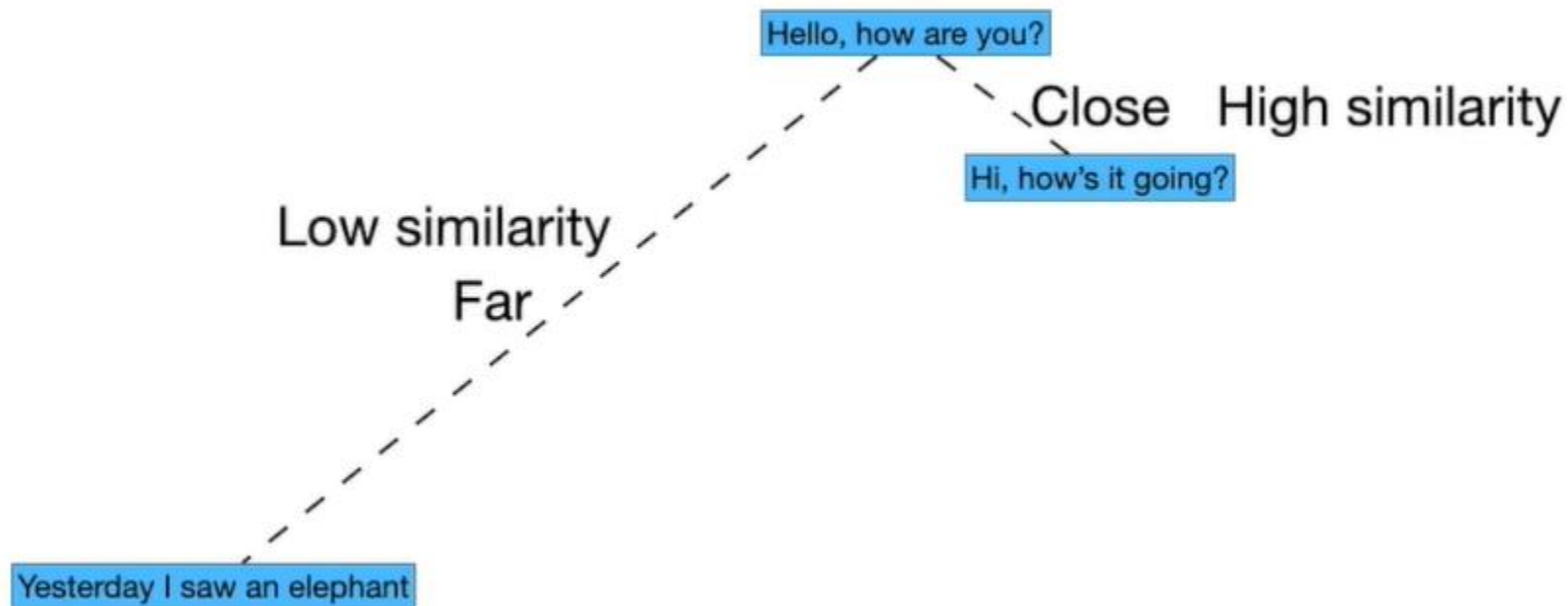


Word	Numbers	
Apple	5	5
Banana	6	5
Strawberry	5	4
Cherry	6	4
Soccer	0	6
Basketball	1	6
Tennis	1	5
Castle	1	2
House	2	2
Building	2	1
Bicycle	5	1
Truck	6	1
Car	6	0

Biểu diễn vec tơ câu



Độ tương tự



Biểu diễn từ dựa trên ngữ cảnh

- **Ý nghĩa của 1 từ được xác định dựa trên ngữ cảnh xuất hiện của nó**
- Để xây dựng biểu diễn cho từ w , ta dựa trên rất nhiều văn bản có chứa từ w

...government debt problems turning into **banking** crises as happened in 2009...
...saying that Europe needs unified **banking** regulation to replace the hodgepodge...
...India has just given its **banking** system a shot in the arm...

These **context words** will represent **banking**

Word vectors

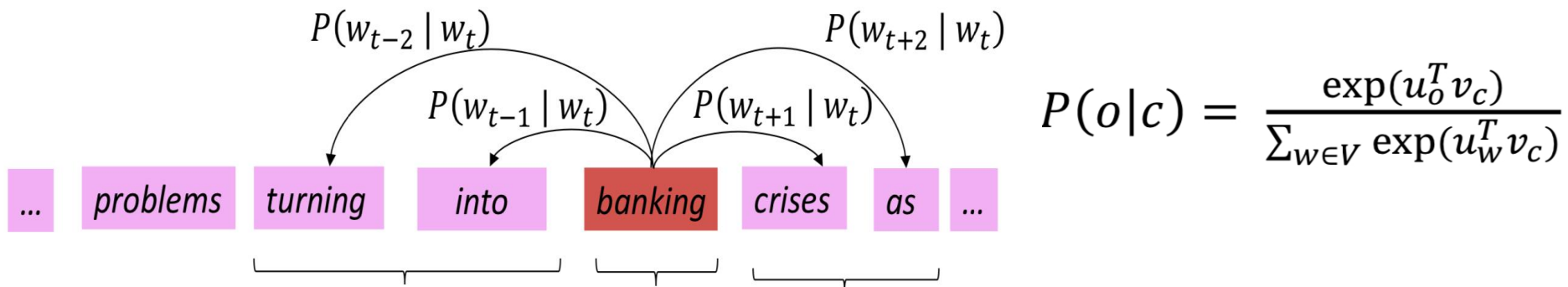
- Mỗi từ được biểu diễn bằng một vector dày đặc (dense vector) sao cho nó các từ tương tự xuất hiện trong các ngữ cảnh tương tự
- Độ tương đồng được đo bằng tích vô hướng của 2 vector

$$\text{banking} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix} \quad \text{monetary} = \begin{pmatrix} 0.413 \\ 0.582 \\ -0.007 \\ 0.247 \\ 0.216 \\ -0.718 \\ 0.147 \\ 0.051 \end{pmatrix}$$

Mô hình word embedding: word2vec

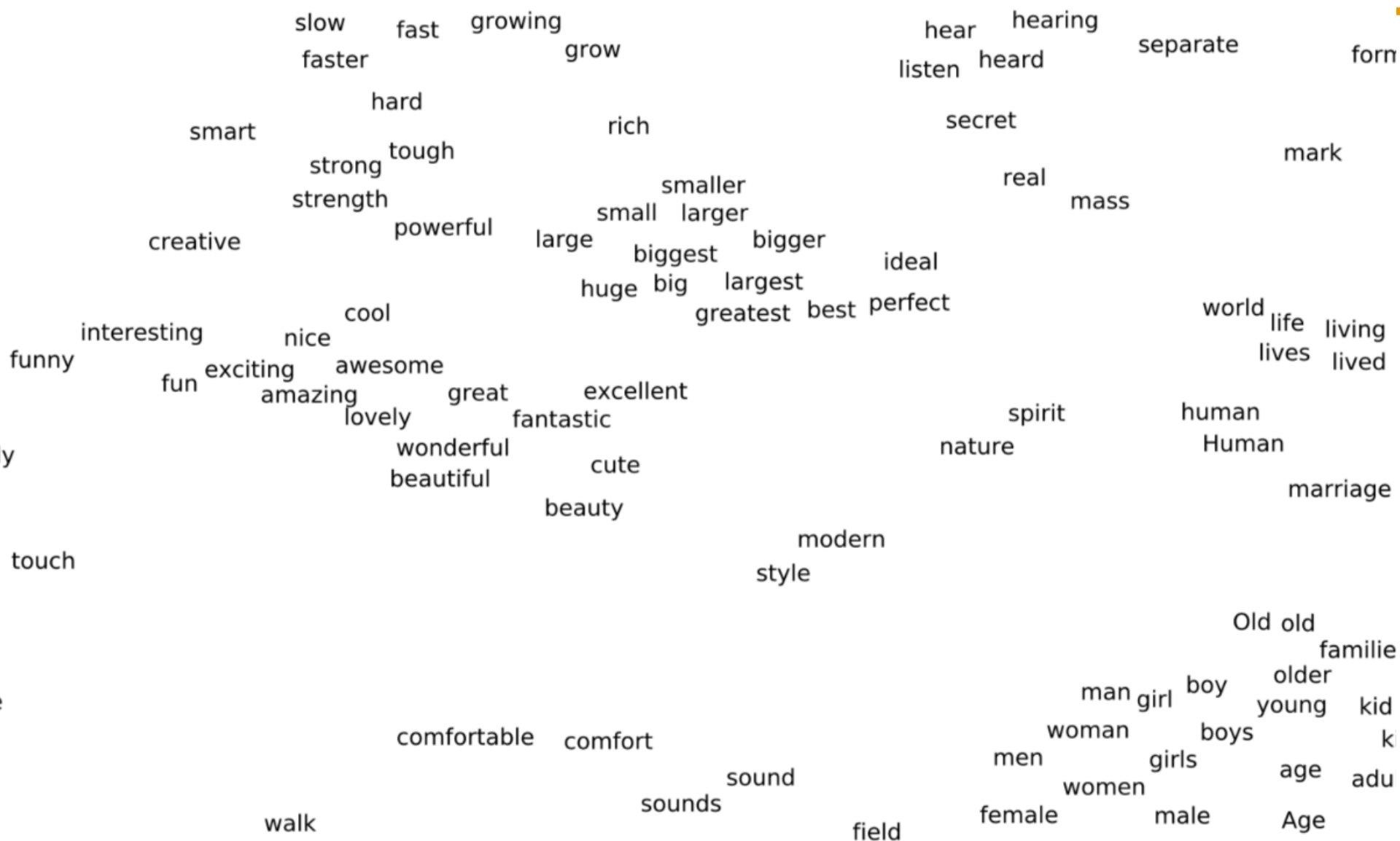
Ý tưởng:

- Khởi tạo từ vec tơ ngẫu nhiên
- Lặp qua mỗi vị trí từ trong toàn bộ tập dữ liệu
- Dự đoán các từ xung quanh sử dụng vec tơ từ

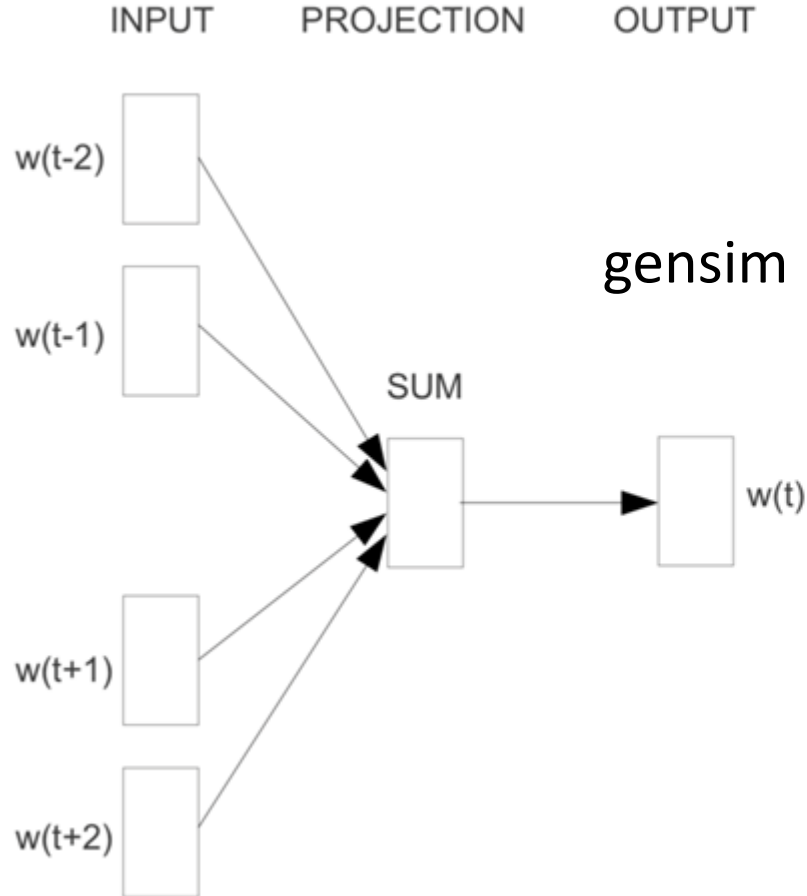


- Học: cập nhật vec tơ sao cho nó dự đoán các từ xung quanh tốt hơn
- Mô hình word embedding cố gắng học các vec tơ sao cho bắt được tốt độ tương đồng của từ và các chiều ngữ nghĩa quan trọng

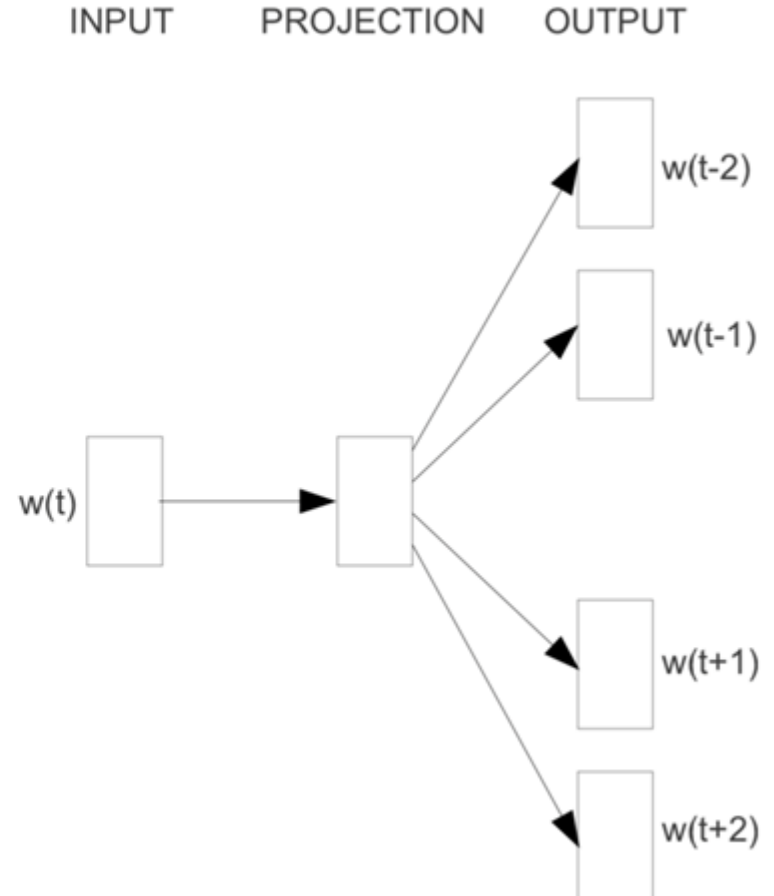
Word2vec tối đa hóa hàm mục tiêu bằng cách đưa các từ tương đồng lại gần nhau trong không gian



Huấn luyện word2vec



CBOW: Dự đoán từ trung tâm khi biết ngữ cảnh



Skip-gram: Dự đoán ngữ cảnh khi biết từ trung tâm

Transformer

- Mô hình học sâu chuyển đổi **chuỗi đầu vào** thành **chuỗi đầu ra** bằng cách sử dụng kiến trúc **encoder – decoder** với:
 - Cơ chế **multi-head attention**
 - Các lớp **feed forward**
 - **Embedding vị trí**
- **Transformer** chỉ dựa vào các cơ chế attention (không sử dụng LSTM). Mô hình Transformer trích xuất đặc trưng cho từng từ bằng cơ chế **self-attention** để xác định mức độ quan trọng của các từ khác trong câu.

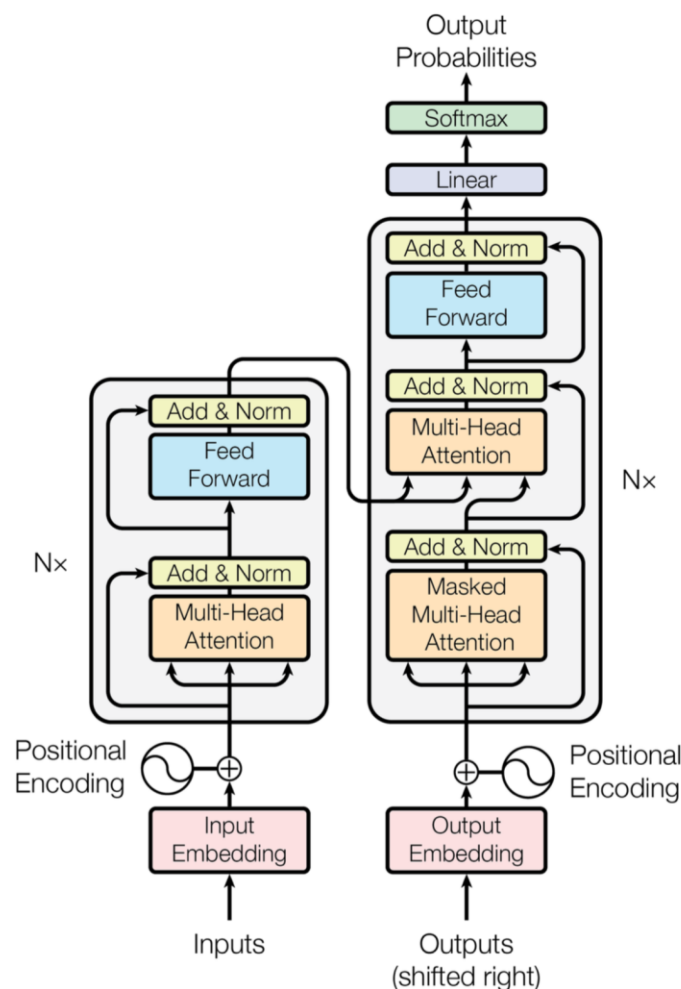
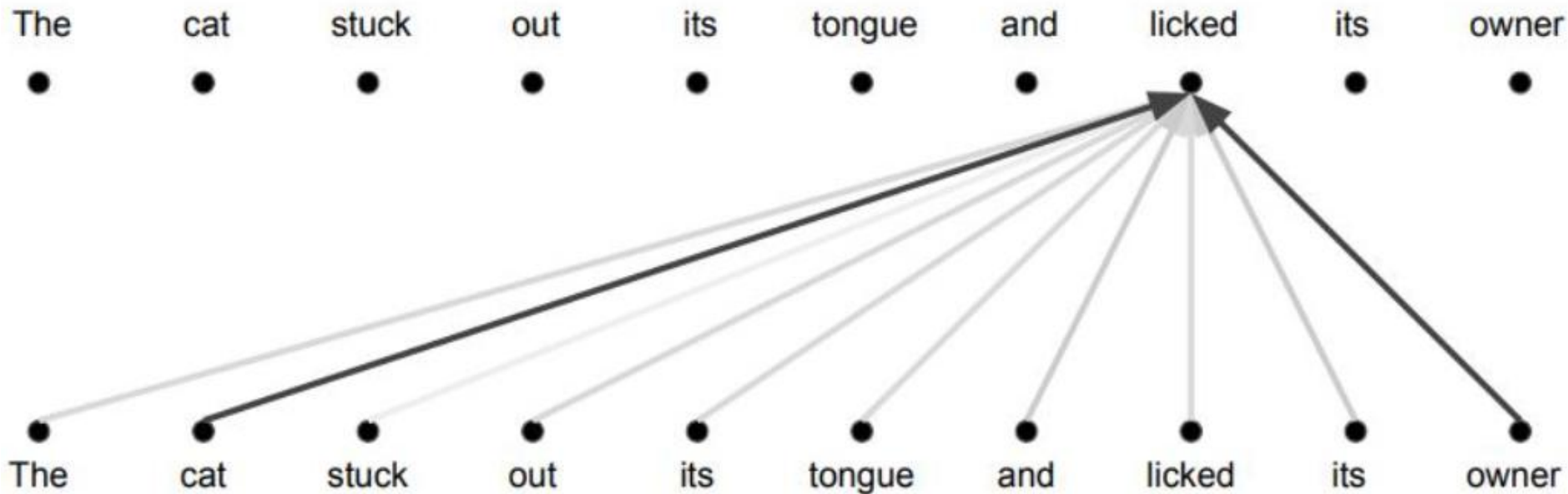


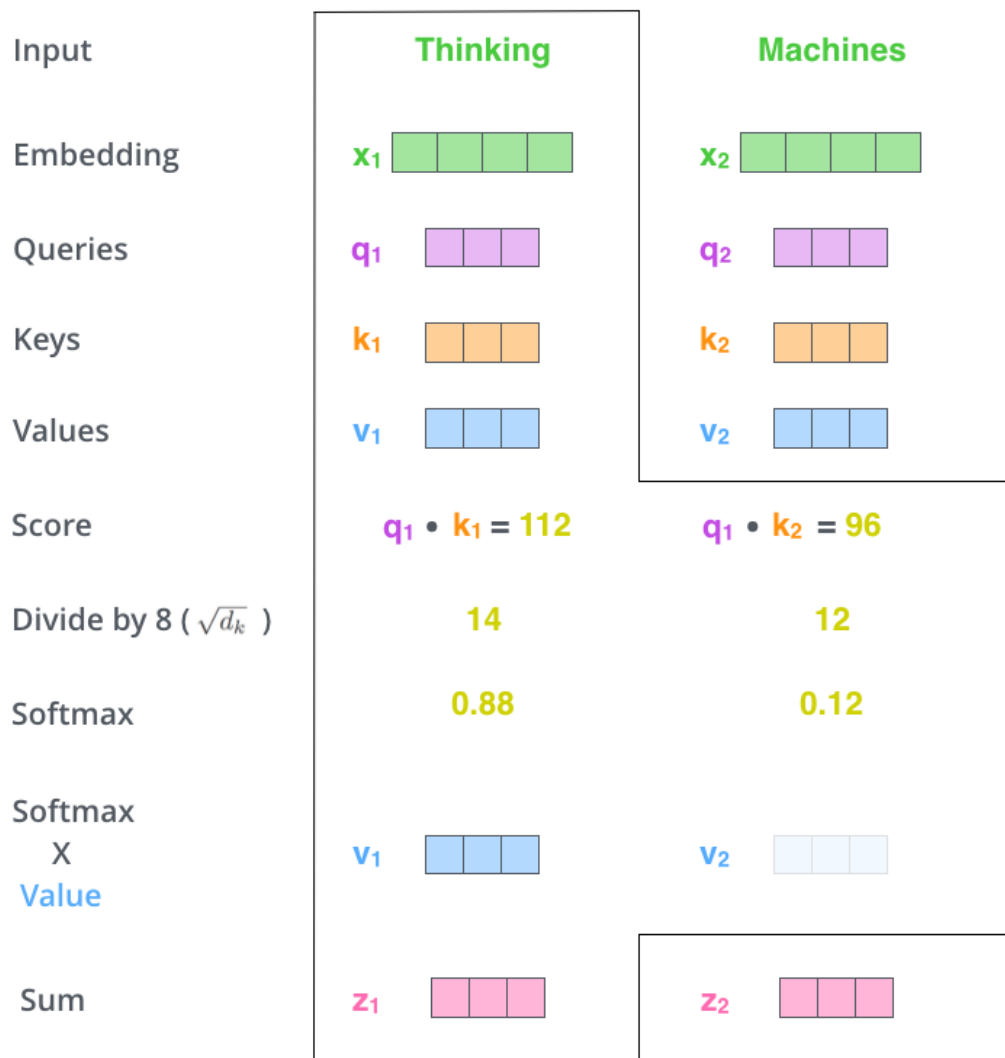
Figure 1: The Transformer - model architecture.

Self-Attention

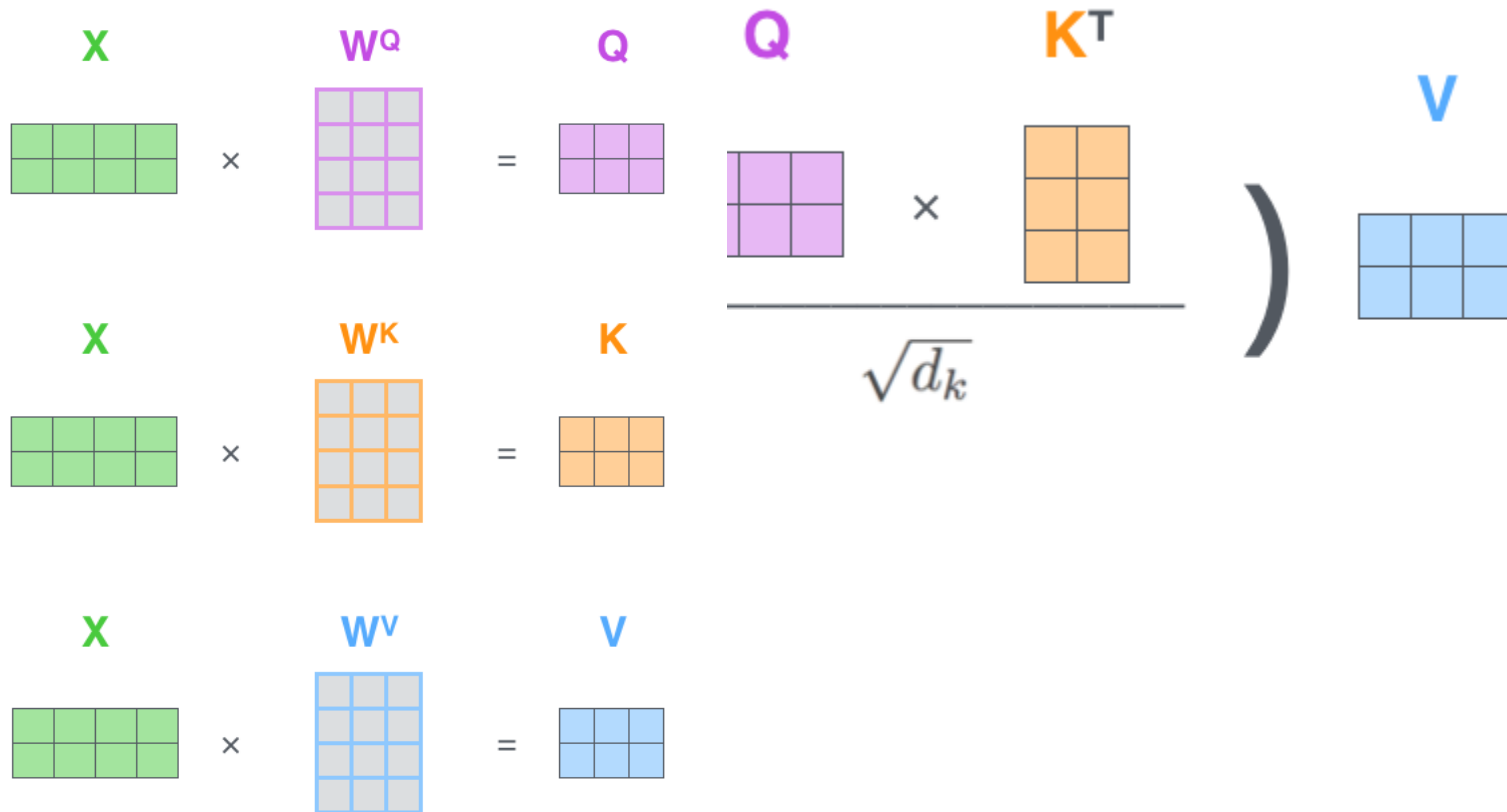
- Có thể song song hóa (theo từng lớp)



Self (Dot-Product) Attention



Self (Dot-Product) Attention



Quá trình tính Attention

- **Mục tiêu:** xác định từ nào cần được chú ý và mức độ chú ý là bao nhiêu

1. Tính Attention Scores:

$$\text{score}(Q_i, K_j) = \frac{Q_i \cdot K_j^\top}{\sqrt{d_k}}$$

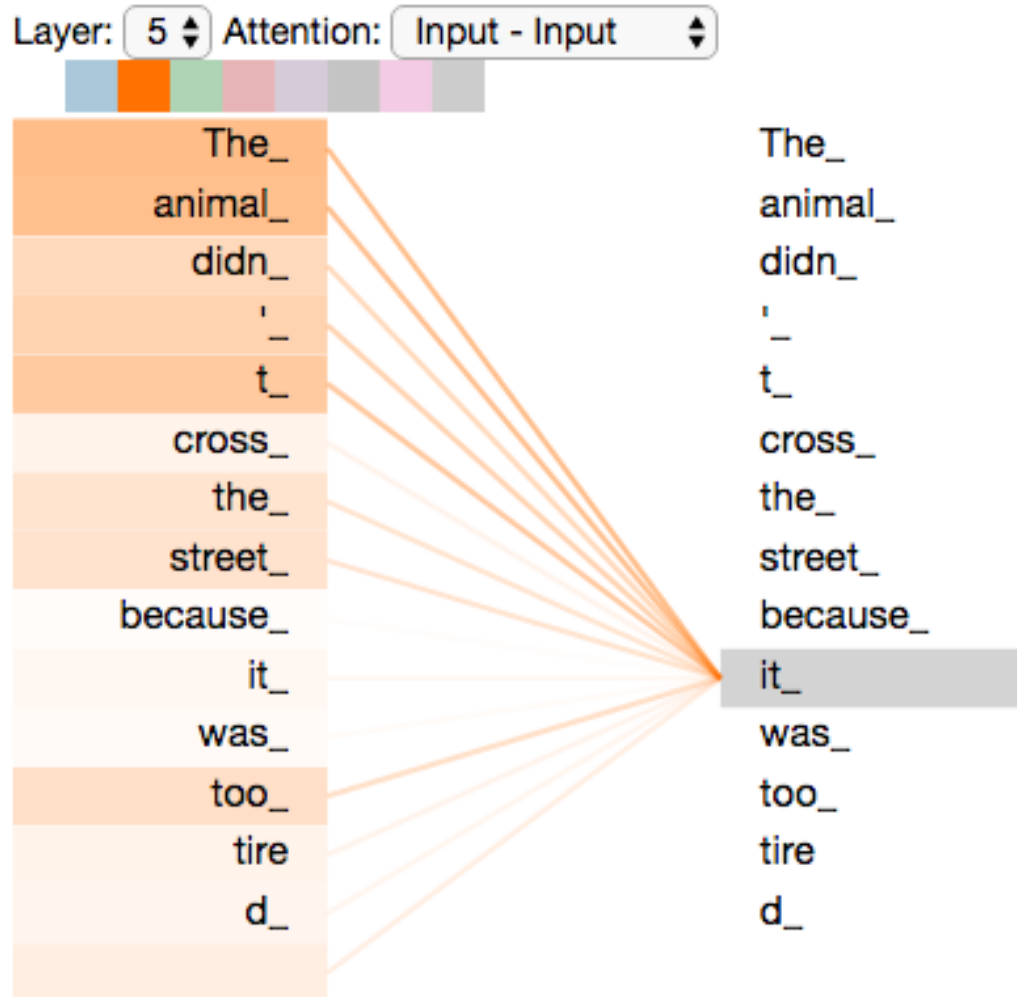
2. Áp dụng hàm Softmax lên các điểm này để chuyển đổi thành xác suất:

$$\text{Attention Weight}_{ij} = \text{Softmax}(\text{score}(Q_i, K_j))$$

3. Kết hợp các Value

$$\text{Output}_i = \sum_j \text{Attention Weight}_{ij} \cdot V_j$$

Minh họa attention ở lớp 5 (trên cùng)



- BERT sử dụng **Transformer**, một cơ chế attention học mối quan hệ ngữ cảnh giữa các từ trong văn bản.
 - **Encoder** đọc đầu vào văn bản, tất cả các từ một lúc
 - **Decoder** để tạo ra dự đoán cho nhiệm vụ.
- Vì mục tiêu của BERT là tạo ra một **mô hình ngôn ngữ**, nên chỉ cần sử dụng **encoder**.

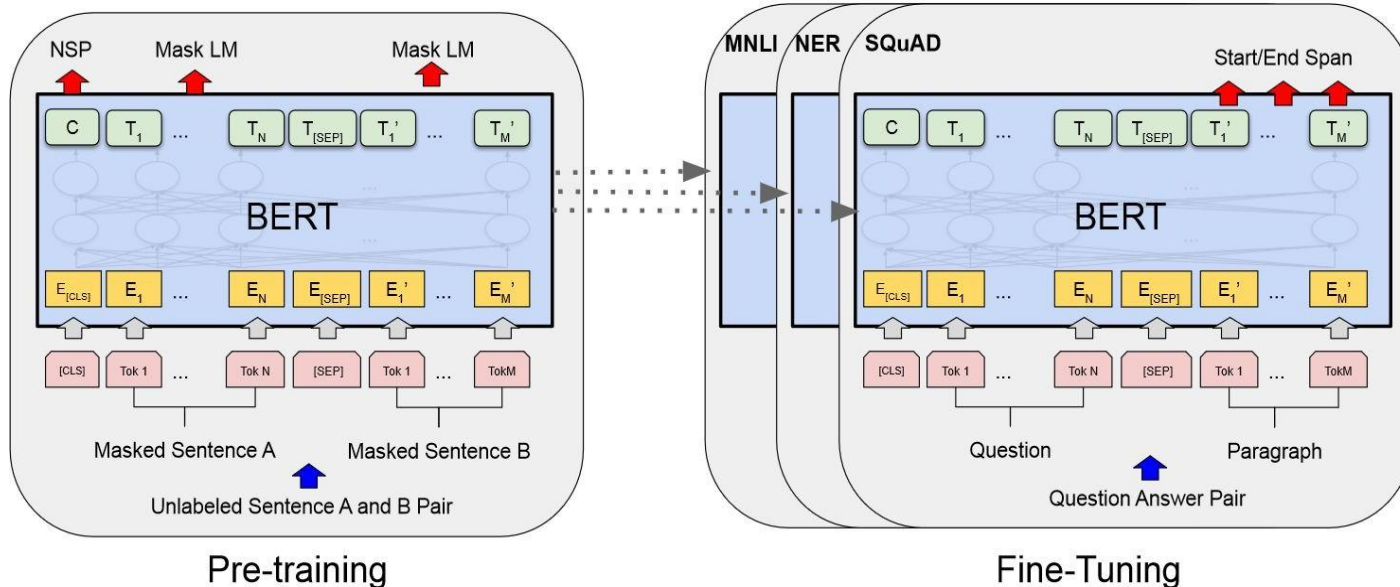
Devlin et al. [BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#), 2019

Pre-training và Fine-tuning

- **[CLS]** là ký hiệu đặc biệt được thêm vào đầu mỗi ví dụ đầu vào.
- **[SEP]** là token đặc biệt dùng để phân tách các đoạn văn.

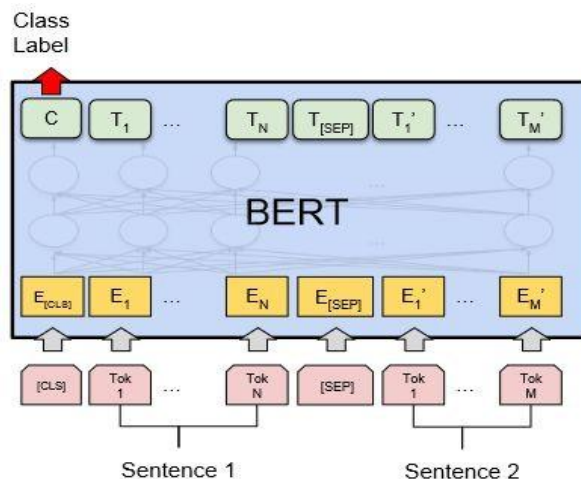
Pre-training

1. **Masked LM:** Che (mask) 15% tổng số token WordPiece trong mỗi chuỗi một cách ngẫu nhiên.
2. **Next Sentence Prediction:** dự đoán 2 câu có phải là 2 câu liên nhau hay không

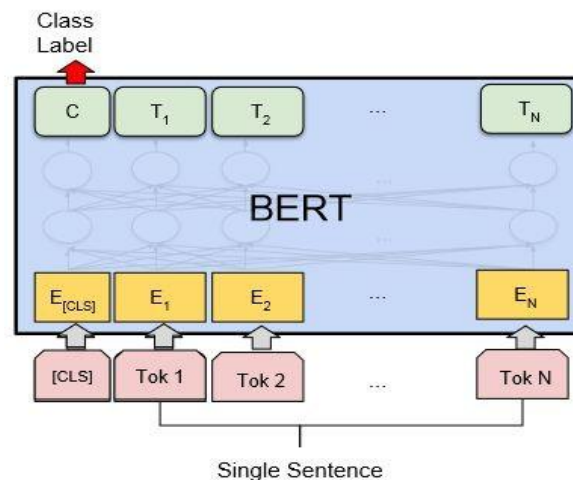


Fine-Tuning

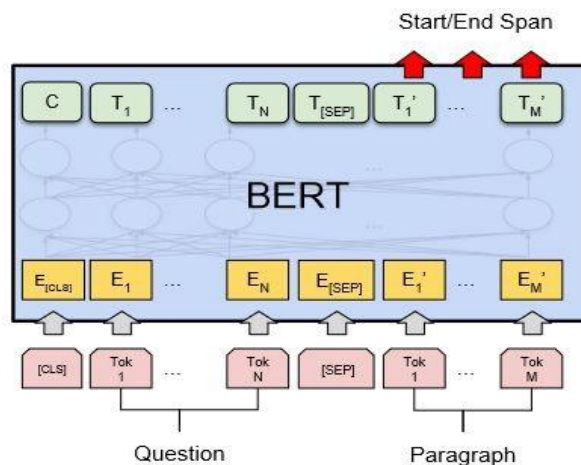
- Fine tuning ít tốn kém hơn pre-training. Được sử dụng cho các downstream task.



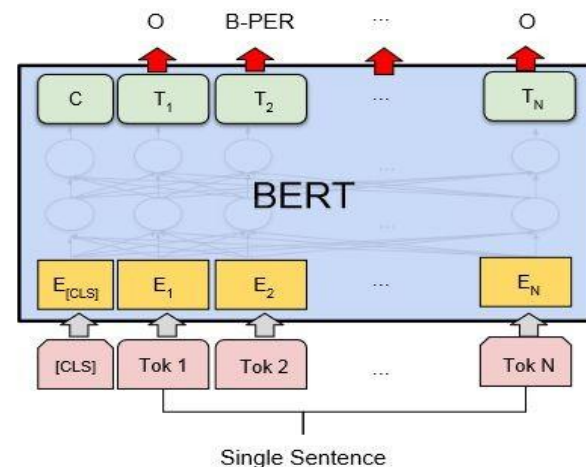
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

Byte-Pair Encoding (BPE) Tokenization

Các bước huấn luyện:

1. Bắt đầu bằng cách tách các từ đầu vào thành các ký tự đơn lẻ (mỗi ký tự tương ứng với một ký hiệu trong từ vựng cuối cùng).
Trong thực tế, thường thêm ký hiệu đặc biệt kết thúc từ " " trước khoảng trắng.
2. Tìm cặp ký hiệu xuất hiện thường xuyên nhất từ từ vựng hiện tại.
3. Thêm cặp này vào từ vựng, làm tăng kích thước từ vựng lên một đơn vị.
4. Lặp lại bước (2) và (3) cho đến khi xây dựng đủ số lượng token đã định hoặc không còn cặp ký hiệu mới nào đạt tần suất yêu cầu.

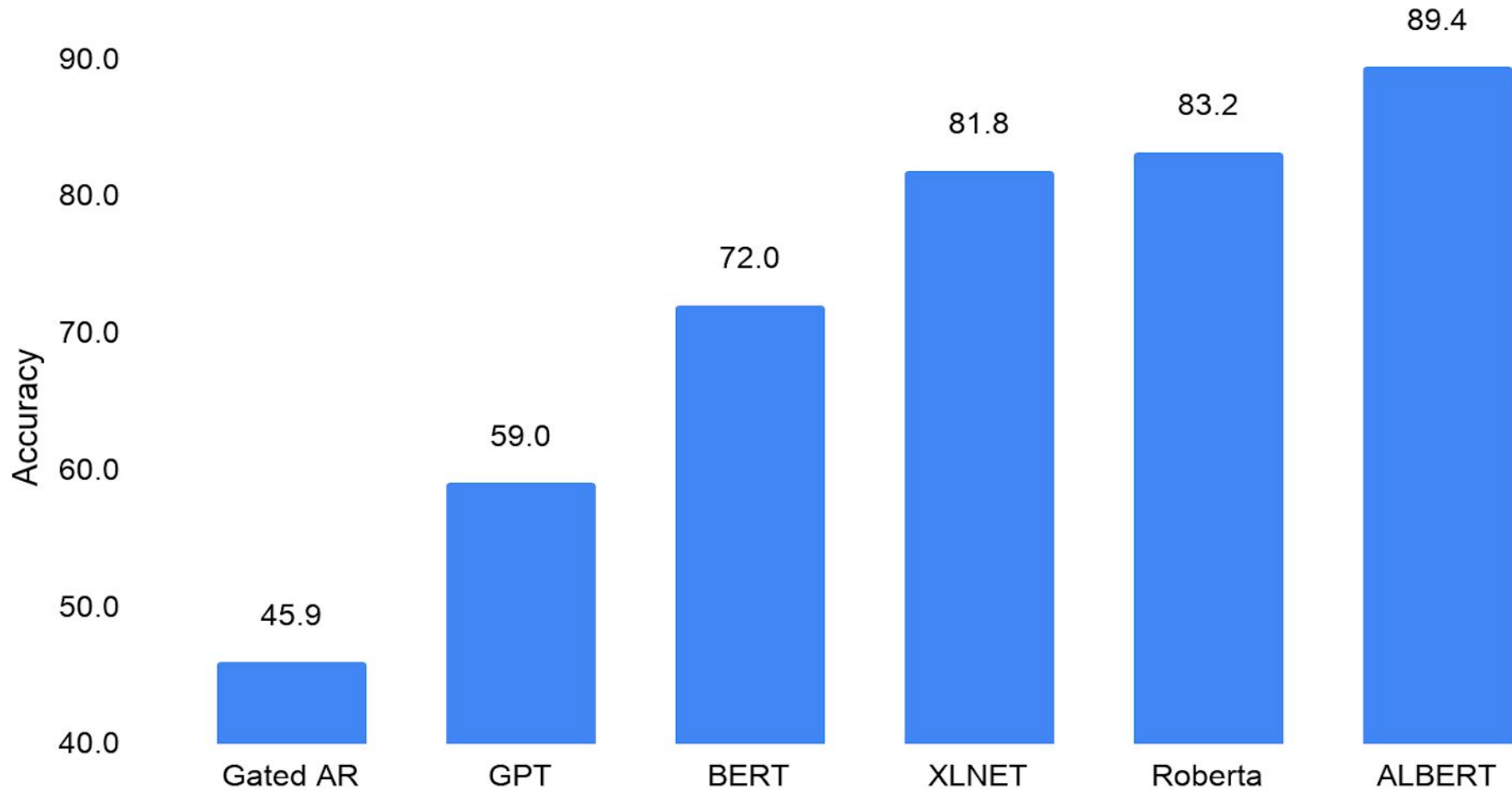
Sennrich et al. [Neural Machine Translation of Rare Words with Subword Units](#), 2016



- **RoBERTa**: Robustly Optimized BERT Approach
 - Facebook AI research team
 - Sử dụng 160GB văn bản thay vì 16GB ban đầu dùng để huấn luyện BERT.
 - Tăng số vòng lặp từ 100K lên 500K.
 - Mẫu masking được thay đổi linh hoạt cho dữ liệu huấn luyện.
 - Không dùng nhiệm vụ Next Sentence Prediction

BERT-based Models

- ALBERT, XLNET, Sentence-BERT, CoBERT

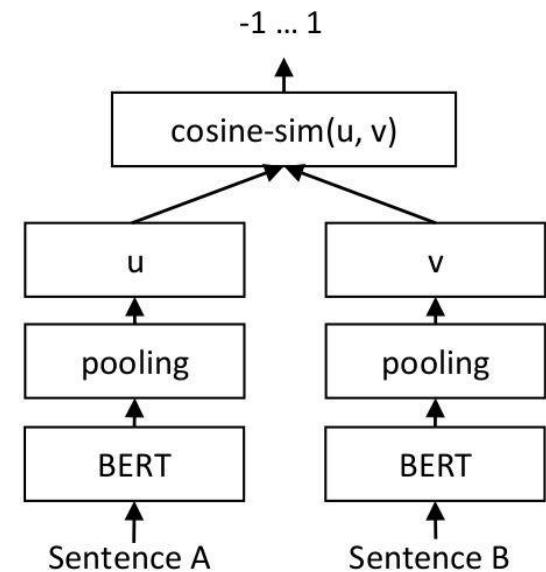


Độ chính xác trong cuộc thi RACE (SAT-like reading comprehension)



Sentence-BERT

- Là biến thể của pretrained BERT, sử dụng cấu trúc mạng **siamese** và **triplet** để tạo ra các **embedding câu**, so sánh bằng **cosine similarity**.
- **SBERT** sử dụng cho các tác vụ mà việc áp dụng BERT không khả thi về mặt tính toán.
- Ví dụ, **phân cụm 10.000 câu** bằng phương pháp phân cụm thứ bậc với BERT mất ~ **65 giờ**, trong khi SBERT chỉ mất **5 giây**.



Reimers and Gurevych. [Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks](https://arxiv.org/abs/1909.01117), 2019

<https://www.sbert.net/>

Mạng Siamese

- Mạng **Siamese** gồm **hai nhánh cấu trúc giống nhau**, dùng để so sánh hai đầu vào nhằm xác định mức độ tương đồng giữa chúng.
- **Cấu trúc và hoạt động:**
 - Đầu vào 2 nhánh có thể là hai câu hoặc hai ảnh
 - Mỗi nhánh sử dụng một mô hình (thường là encoder) để biến đầu vào thành **vector embedding**.
 - Kết quả embedding từ hai nhánh được so sánh bằng một **hàm khoảng cách** (như **cosine similarity** hoặc **Euclidean distance**).
 - Đầu ra là giá trị **độ tương đồng** giữa hai đầu vào.

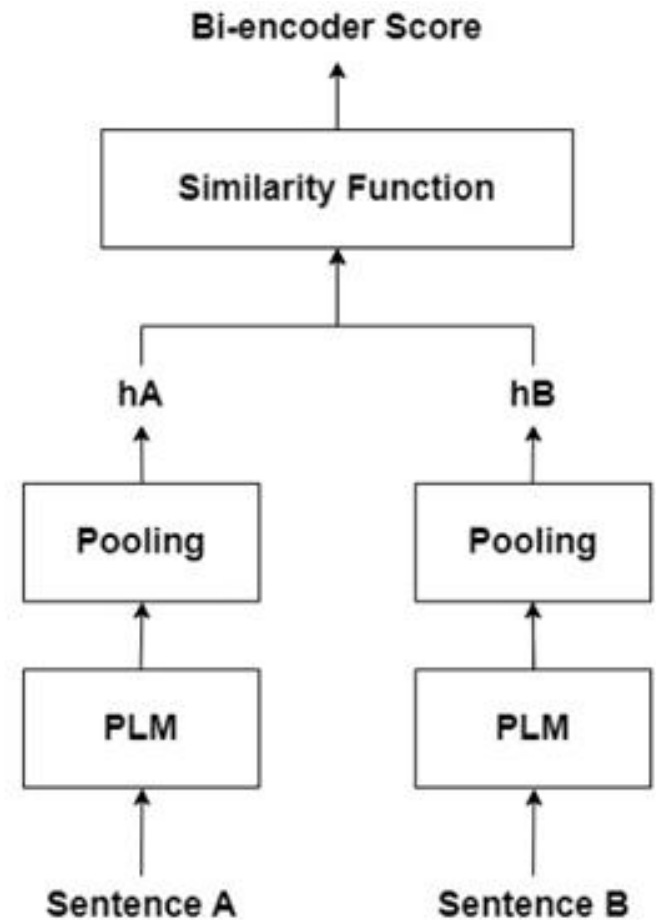
Mạng Triplet

- Mạng Triplet mở rộng từ mạng Siamese và dùng ba đầu vào để học sự khác biệt giữa các mẫu tương đồng và không tương đồng.
- Cấu trúc và hoạt động:
 - Ba đầu vào: mẫu chính (Anchor), mẫu positive, mẫu negative
 - Mỗi đầu vào được đưa qua một encoder để tạo ra các embedding vector.
 - Mục tiêu huấn luyện: Tối ưu hóa sao cho:
$$d(\text{Anchor}, \text{Positive}) + \text{margin} < d(\text{Anchor}, \text{Negative})$$
$$d$$
 là khoảng cách, ví dụ: Euclidean distance

- **Sentence Transformer**: framework được xây dựng trên nền SBERT, mở rộng với nhiều loại mô hình và ứng dụng.
 - Hỗ trợ BERT, RoBERTa, DistilBERT, MiniLM, và các mô hình đa ngôn ngữ.
 - Cung cấp môi trường dễ sử dụng với các công cụ tiện lợi để thực hiện các bài toán như:
 - **Semantic Search** (Tìm kiếm ngữ nghĩa)
 - **Text Similarity** (Tính độ tương tự văn bản)
 - **Clustering** (Phân cụm văn bản)

Bi encoder

- Là một kiến trúc mạng neural encoder trong các mô hình Transformer (như BERT, RoBERTa, Sentence Transformer).
- Được thiết kế để biểu diễn các đoạn văn bản thành vector embedding, giúp tối ưu hóa cho các bài toán tìm kiếm ngữ nghĩa

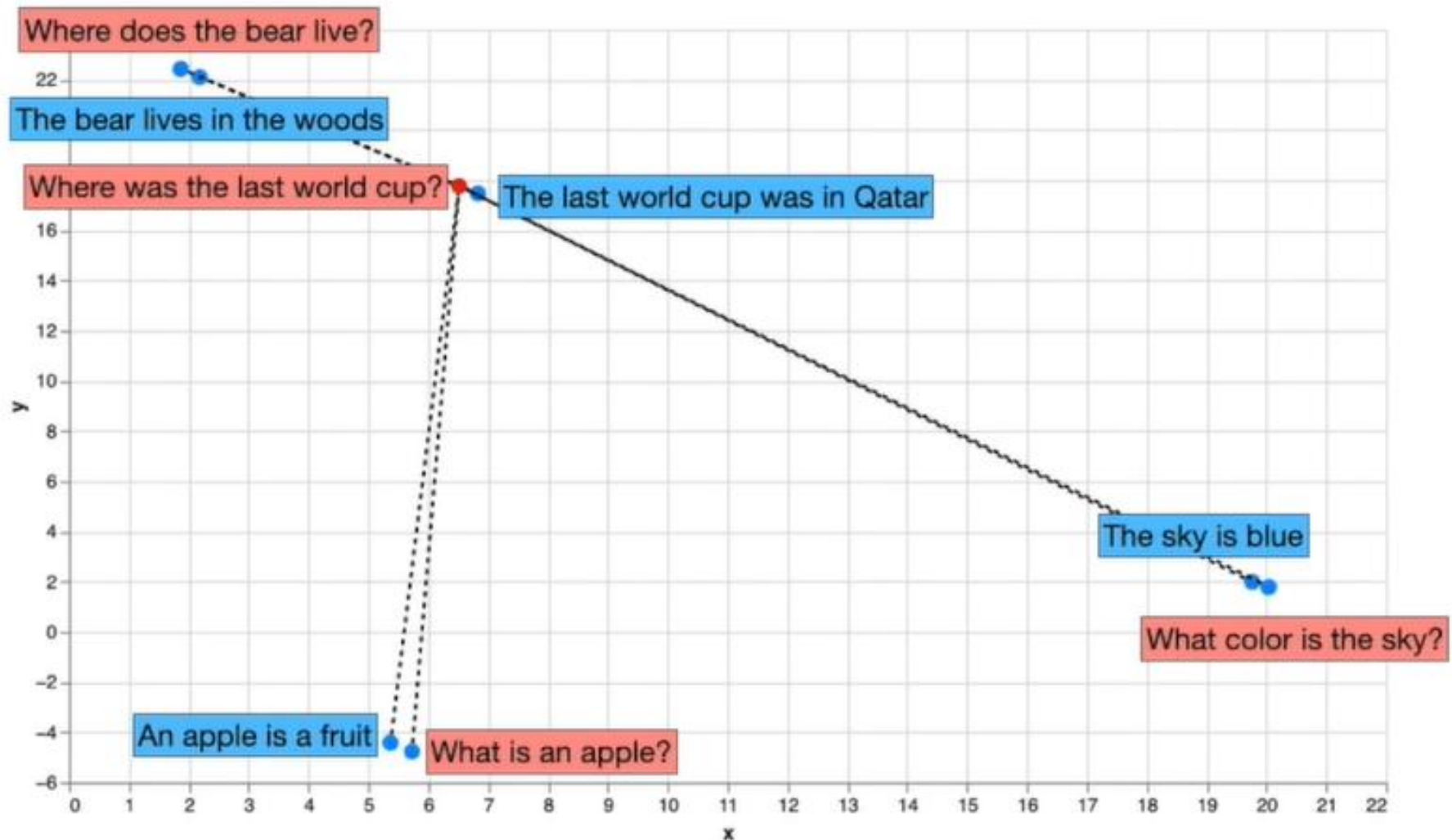


Example

```
pip install sentence-transformers
```

```
from sentence_transformers import SentenceTransformer, util
# Tải mô hình bi-encoder từ Sentence Transformer
model = SentenceTransformer('all-MiniLM-L6-v2')
# Danh sách các tài liệu
documents = [
    "The capital of France is Paris.",
    "Machine learning enables computers to learn from data.",
    "Artificial intelligence is the future.",
    "Paris is known for its art and culture."
]
# Mã hóa các tài liệu thành vector embedding
doc_embeddings = model.encode(documents, convert_to_tensor=True)
# Câu truy vấn
query = "What is the capital of France?"
query_embedding = model.encode(query, convert_to_tensor=True)
# Tính độ tương tự cosine giữa câu truy vấn và các tài liệu
scores = util.cos_sim(query_embedding, doc_embeddings)
# Tìm tài liệu có độ tương tự cao nhất
best_match_idx = scores.argmax().item()
print(f"Tài liệu phù hợp nhất: {documents[best_match_idx]}")
```

Nearest neighbors: good but slow



Solution: Inverted index file

Các bước xây dựng:

1. Tạo Embedding:

- Mỗi tài liệu/truy vấn được chuyển sang dạng vector sử dụng **pre-trained language model**, vd BERT, Sentence Transformers, hoặc PhoBERT.
- Ví dụ: Câu "apple is a fruit" chuyển thành vector [0.1, 0.3, 0.7, ...].

2. Gán **Semantic Tokens (Quantization)**:

- Vì embedding là dữ liệu liên tục, cần được lượng tử hóa hoặc phân cụm để có thể tìm kiếm.
- Có thể sử dụng ANN (Approximate Nearest Neighbors) như HNSW, hoặc các kỹ thuật lượng tử hóa vector để ánh xạ vector thành mã hoặc cụm rời rạc.
- Ví dụ: Một embedding có thể được gán cho **cluster ID 42** hoặc **semantic token** đại diện cho các vector tương tự

3. Xây dựng Chỉ mục Ngược:

- Thay vì lập chỉ mục cho từ, chỉ mục này sẽ ánh xạ token ngữ nghĩa hoặc ID cụm đến các tài liệu chứa chúng.
- Ví dụ
 - Cluster 42 → [Doc A, Doc C, Doc F]
 - Cluster 51 → [Doc B, Doc D]

4. Lưu trữ Metadata:

- (tùy chọn) lưu metadata (ví dụ: vector norms và top-k embeddings) để tăng tốc quá trình tính độ tương đồng khi truy xuất.

Bước truy vấn:

1. Tạo Embedding cho Truy vấn:

- Khi người dùng gửi truy vấn (ví dụ: "find fruits related to apple"), truy vấn này được chuyển thành **vector embedding**.

2. Lượng tử hóa / Tìm Hàng xóm Gần nhất:

- Vector truy vấn được ánh xạ tới **ID cụm** hoặc **semantic tokens** gần nhất trong **chỉ mục ngược**.
- **Ví dụ:** Query embedding → Cluster 42 (nơi chứa các embedding liên quan đến "fruit").

3. Tìm kiếm trong Chỉ mục Ngược:

- Hệ thống truy xuất các tài liệu tương ứng với các cụm liên quan (ví dụ: "Cluster 42 → [Doc A, Doc C, Doc F]").

4. Xếp hạng Kết quả:

- Cuối cùng, các tài liệu được **xếp hạng lại** dựa trên **cosine similarity** giữa **vector truy vấn** và **embedding của tài liệu**.

Các phương pháp phân cụm trong Dense Retrieval

- Mục tiêu: Giảm chi phí tìm kiếm trên hàng triệu vector bằng cách chia không gian ngữ nghĩa thành các cụm (clusters).
- Các phương pháp phân cụm:
 - K-Means Chuẩn
 - Mini-batch K-Means
 - Hierarchical Clustering.
 - Product Quantization (PQ) / Inverted File (IVF)
 - Incremental / Online Clustering

1. Chọn ngẫu nhiên k điểm làm **tâm cụm ban đầu** (centroids).
2. Với mỗi điểm dữ liệu x_i , tính khoảng cách đến tất cả các tâm cụm.
3. Gán x_i vào cụm có tâm gần nhất.
4. Sau khi gán xong, cập nhật tâm cụm c_j bằng **trung bình vector** của tất cả điểm trong cụm j :

$$c_j = \frac{1}{|S_j|} \sum_{x_i \in S_j} x_i$$

5. Lặp bước 2–3 cho đến khi tâm cụm hội tụ (không thay đổi nhiều) hoặc đạt số vòng lặp tối đa.
- Thuật toán cố gắng **tối thiểu hóa tổng bình phương khoảng cách** giữa các điểm và tâm cụm:

$$J = \sum_{j=1}^k \sum_{x_i \in S_j} \|x_i - c_j\|^2$$

Các phương pháp phân cụm

- **Mini-batch K-Means:**

- Cập nhật centroid dựa trên **mini-batch** mỗi vòng.
- Giảm mạnh chi phí huấn luyện.
- Thời gian: nhanh hơn 10–100×
- Bộ nhớ: thấp
- Gần chính xác như k-means chuẩn

- **Hierarchical Clustering:**

- Phân cụm nhiều tầng: từ cụm lớn → cụm nhỏ.
- Dễ mở rộng, thích hợp cho dữ liệu khổng lồ.
- Thời gian: Trung bình
- Bộ nhớ: Trung bình
- Cụm ổn định, có cấu trúc cây

Các phương pháp phân cụm

- **Product Quantization (PQ) / Inverted File (IVF):**

- Chia không gian vector thành nhiều ô (**cells**) nhỏ bằng lượng tử hóa.
- Mỗi ô đóng vai trò như một cụm.
- Dùng trong FAISS, Milvus, ScaNN.
- Thời gian: Rất nhanh
- Bộ nhớ: Rất thấp (nén vector)

- **Incremental / Online Clustering:**

- Cập nhật cụm dần khi có dữ liệu mới.
- Không cần tái huấn luyện toàn bộ.
- Thời gian: Liên tục
- Bộ nhớ: Linh hoạt
- Dễ bảo trì hệ thống lớn

- Các hệ thống thực tế (FAISS, Milvus, Weaviate) kết hợp Mini-batch K-Means + PQ/IVF để đạt tốc độ và hiệu quả cao.

Hierarchical Navigable Small-World Graphs

- **HNSW** là thuật toán lập chỉ mục và tìm kiếm xấp xỉ lân cận gần nhất (Approximate Nearest Neighbor - ANN) dùng để tìm kiếm các phần tử gần nhất trong không gian vector một cách nhanh chóng và hiệu quả.
- Cấu trúc đồ thị phân cấp
- Xây dựng đồ thị:
 - Khi thêm một điểm dữ liệu mới, HNSW chèn nó vào các lớp từ trên xuống dưới.
 - Ở mỗi lớp, thuật toán chọn các **hàng xóm gần nhất** và tạo liên kết giữa chúng.
- Khi tìm kiếm, thuật toán bắt đầu từ lớp cao nhất và dần dần chuyển xuống các lớp thấp hơn, mỗi lần điều hướng đến hàng xóm gần nhất cho đến khi tìm ra điểm gần nhất mong muốn.

Comparison Approximate Nearest Neighbor Methods

Feature	HNSW	Faiss	LSH (Locality Sensitive Hashing)
Accuracy	High	Medium	Lower
Speed	Fast	Very Fast	Fast
Incremental Update	Yes	No	Yes
Index Size	Moderate	Moderate to Large	Small

Example

Cho các tài liệu:

"The capital of France is Paris.",

"The tallest mountain in the world is Mount Everest.",

"Python is a popular programming language.",

"AI is transforming the world.",

"Blockchain is a distributed ledger technology."

Cho query:

"What is the capital of France?"

Viết chương trình thực hiện các việc sau:

1. Biểu diễn các tài liệu đầu vào thành dạng vector
2. Index các tài liệu đó
3. Tìm kiếm tài liệu thỏa 1 câu truy vấn cho trước, sử dụng bi-encoder

Example

```
!pip install faiss-cpu sentence-transformers
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer
```

1. Biểu diễn tài liệu và câu truy vấn thành vector

```
model = SentenceTransformer('paraphrase-MiniLM-L6-v2') # Bi-encoder
sẵn có, có thể thay bằng PhoBERT nếu cần
# Dữ liệu tài liệu (một danh sách các văn bản)
documents = [
    "The capital of France is Paris.",
    "The tallest mountain in the world is Mount Everest.",
    "Python is a popular programming language.",
    "AI is transforming the world.",
    "Blockchain is a distributed ledger technology."
]
```

Mã hóa các tài liệu thành vector embedding

```
doc_embeddings = model.encode(documents, convert_to_tensor=False)
```

Example

2. Index các tài liệu bằng FAISS

```
d = doc_embeddings.shape[1] # Kích thước vector embedding
index = faiss.IndexFlatL2(d) # Sử dụng IndexFlatL2 với khoảng cách
Euclidean
index.add(np.array(doc_embeddings, dtype='float32')) # Thêm các
vector vào index
print(f"Đã index {index.ntotal} tài liệu.")
```

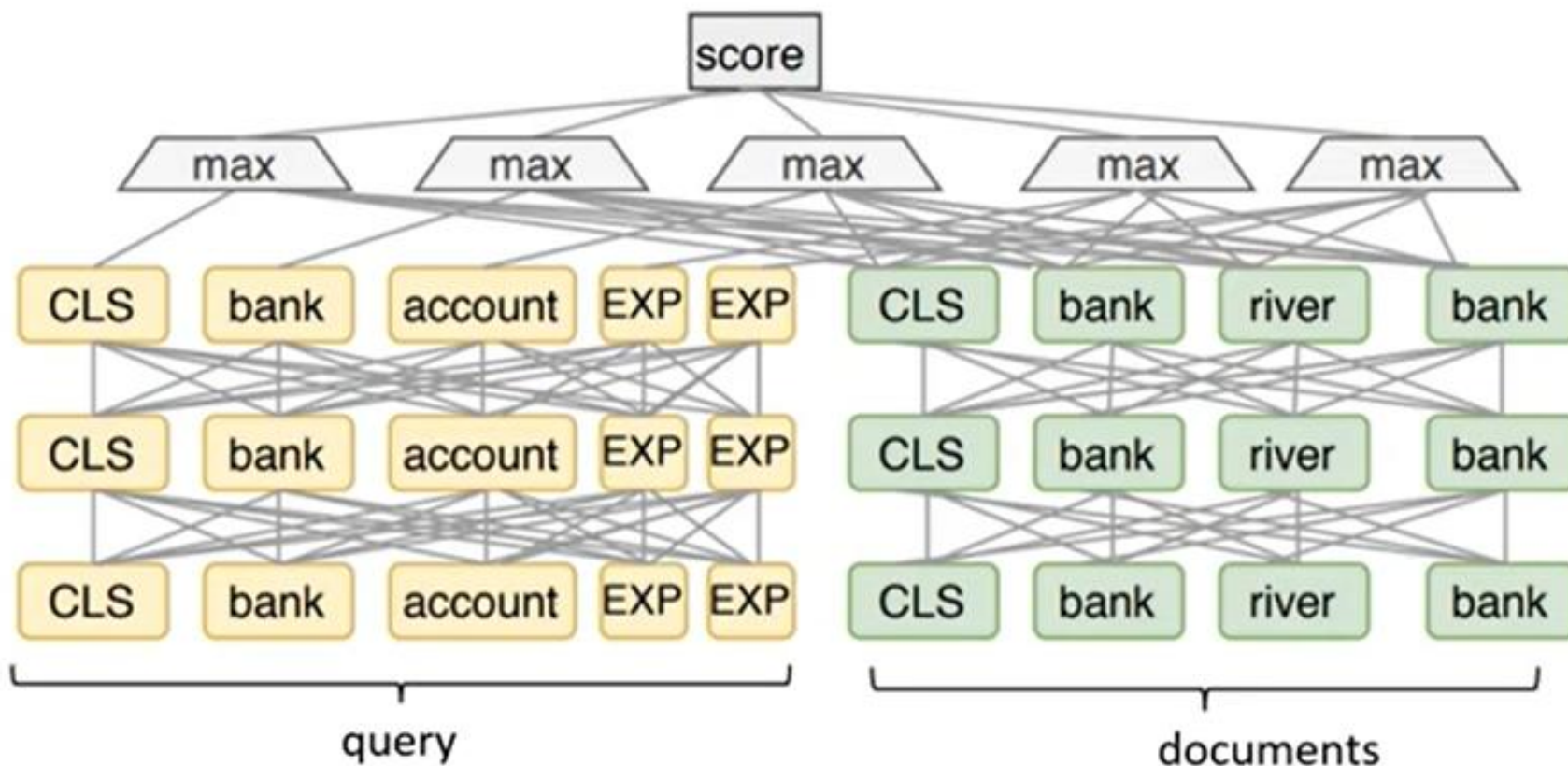
3. Tìm kiếm tài liệu dựa trên câu truy vấn

```
query = "What is the capital of France?" # Câu truy vấn
query_embedding = model.encode(query,
convert_to_tensor=False).reshape(1, -1) # Mã hóa câu truy vấn
k = 3 # Số lượng tài liệu gần nhất cần tìm
distances, indices = index.search(np.array(query_embedding,
dtype='float32'), k)
```

4. In kết quả tìm kiếm

```
print("\nKết quả tìm kiếm:")
for i, idx in enumerate(indices[0]):
    print(f"Tài liệu {i+1}: {documents[idx]} (Khoảng cách:
{distances[0][i]:.4f})")
```

- **CoIBERT** (Contextualized Late Interaction over BERT): tạo ra vector embedding cho từng token.



Khattab and Zaharia, [Colbert: Efficient and effective passage search via contextualized late interaction over BERT](#), 2020

Hoạt động:

- Cả truy vấn và tài liệu được đưa qua BERT để sinh ra embedding cho từng token.
- Tương tác muộn (Late Interaction):
 - Khi tính độ tương tự giữa truy vấn và tài liệu, CoBERT không so sánh toàn bộ embedding mà sử dụng các phép max-sim, cho phép chỉ lấy giá trị tương tự cao nhất giữa từng cặp token liên quan.
- Tính tổng điểm tương tự:
 - Sau khi tìm độ tương tự lớn nhất giữa các token của truy vấn và tài liệu, các điểm số được cộng lại để tạo thành điểm tổng hợp cho toàn bộ tài liệu.

- **Ưu điểm:**

- **Nhanh và chính xác:** Kết hợp giữa mã hóa độc lập (nhANH) và tương tác ngữ cảnh (chính xác).
- **Khả năng mở rộng:** Phù hợp cho các hệ thống tìm kiếm quy mô lớn.
- **Tích hợp dễ dàng với FAISS** để tăng tốc độ tìm kiếm.

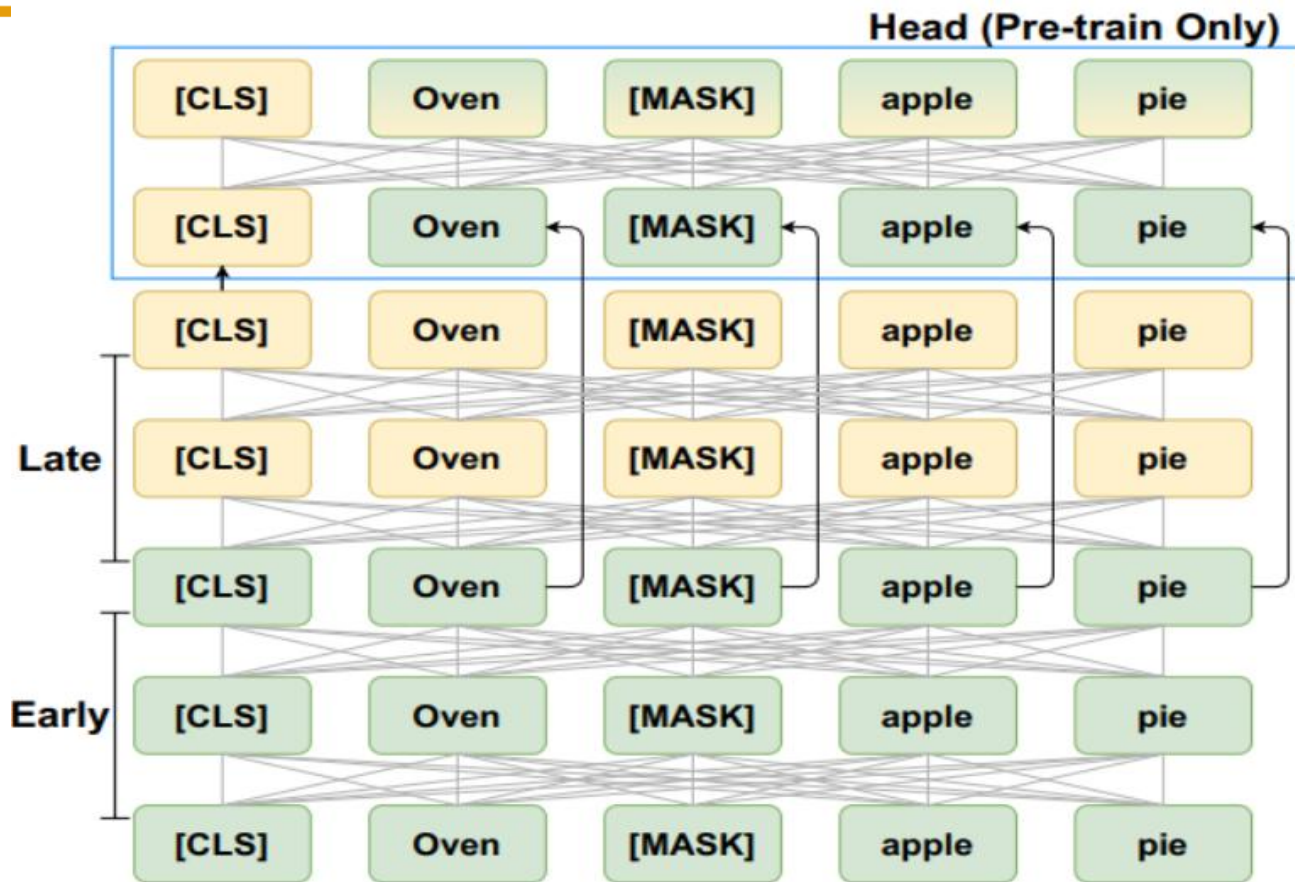
- **Nhược điểm:**

- **Tốn không gian lưu trữ:** Vì cần lưu trữ embedding cho từng token của tài liệu.
- **Phức tạp hơn bi-encoder:** Do cần tính toán tương tác giữa các token trong quá trình tìm kiếm.

Condenser

- BERT không tối ưu cho retrieval; ColBERT có thể gặp vấn đề về tốc độ và tài nguyên.
- Condenser khắc phục những hạn chế này bằng cách cô đọng thông tin quan trọng hơn từ tài liệu.

Condenser



- Nhờ quá trình Pretrained đặc biệt, khả năng mã hóa tài liệu nhanh, Condenser là lựa chọn lý tưởng cho các hệ thống semantic search quy mô lớn.

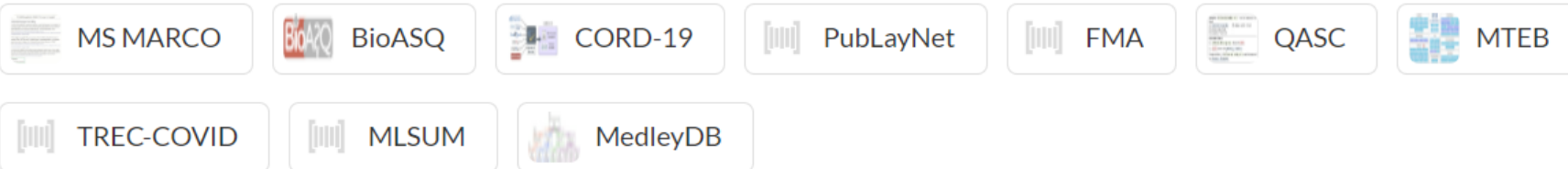
Hoạt động:

- Chiến lược Pretrained kết hợp thông tin từ nhiều tầng trong mạng → cô đọng thông tin ngữ nghĩa quan trọng vào vector embedding cuối cùng.
- Condenser hoạt động tốt trong các hệ thống bi-encoder. Khi tích hợp với FAISS, nó cho phép mã hóa tài liệu nhanh chóng và hiệu quả hơn



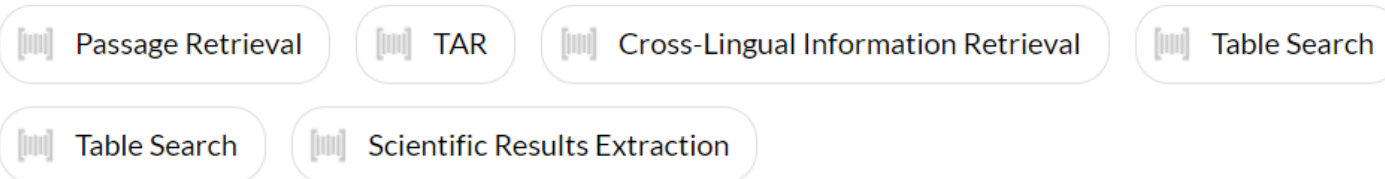
paperswithcode.com/task/information-retrieval

Datasets



[See all 88 information retrieval datasets](#)

Subtasks



- **Nguồn dữ liệu phong phú:** Các câu hỏi được lấy từ lịch sử tìm kiếm thực của người dùng Bing, phản ánh nhu cầu thực tế.
- **Tính đa dạng về ngữ cảnh:** Câu hỏi và tài liệu đi kèm thuộc nhiều chủ đề khác nhau như y tế, khoa học, du lịch, văn hóa, v.v.
- **Khó khăn:** Dữ liệu có thể không rõ ràng hoặc chứa nhiều thông tin nhiễu, yêu cầu hệ thống phải hiểu ngôn ngữ tự nhiên ở mức sâu.

Các tác vụ trong MS-MARCO

- **Passage Ranking:** Xếp hạng đoạn văn dựa trên truy vấn.
 - Quy mô: Khoảng 8.8 triệu đoạn văn từ các trang web phổ biến.
- **Document Ranking:** Xếp hạng tài liệu dựa trên truy vấn.
 - Quy mô: 3.2 triệu tài liệu.
 - Khó khăn: Xử lý tài liệu yêu cầu hệ thống phải chọn lọc thông tin một cách hiệu quả.
- **Question Answering:** Trả lời câu hỏi từ đoạn văn hoặc tài liệu được cung cấp.
 - Quy mô: 100,000 câu hỏi từ người dùng thực tế trên công cụ tìm kiếm Bing, kèm theo các câu trả lời ngắn được trích xuất.
- **Conversational Search:** Tìm kiếm hội thoại liên tục, trong đó mỗi truy vấn có ngữ cảnh từ các câu hỏi trước.
 - Tính năng : Xử lý ngữ cảnh liên phiên để cung cấp câu trả lời chính xác hơn.

BM42: New Baseline for Hybrid Search

- BM42 khắc phục hạn chế của BM25 và các mô hình hiện đại dựa trên transformer như SPLADE.
 - Kết hợp khả năng tìm kiếm vector thưa của BM25 với sức mạnh của các mô hình transformer, tận dụng ma trận attention để đánh giá mức độ quan trọng của các từ trong tài liệu.
 - So với SPLADE, BM42 vượt trội nhờ hiệu quả tính toán cao hơn, tiết kiệm bộ nhớ hơn và xử lý token đơn giản hơn.
-
- [BM42: New Baseline for Hybrid Search – Qdrant](#)
 - [Qdrant Unveils BM42: A Cutting-Edge Pure Vector-Based Hybrid Search Algorithm Optimizing RAG and AI Applications - MarkTechPost](#)
 - [hybrid_retrieval_bm42.ipynb – Colab](#)
 - [GitHub - qdrant/bm42_eval: Evaluation of bm42 sparse indexing algorithm](#)



Sparse vector search – Nhắc lại BM25

$$\text{score}(D, Q) = \sum_{i=1}^N \text{IDF}(q_i) \times \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

Term importance with Corpora

Term importance with Document

$$\text{IDF}(q_i) = \frac{\text{Number of documents}}{\text{Number of documents with } q_i}$$

- $f(q_i, D)$: Frequency of term q_i in document D , representing the number of times q_i appears in D .
- k_1 and b : Hyperparameters, typically set to 1.5 and 0.75, influencing the effect of term frequency and document length.
- $|D| / \text{avgdl}$: The document length relative to the average length in the corpus. If a token appears in a shorter document, it is likely to be more important for that document.

BM25



Longer document: web page, paper, books

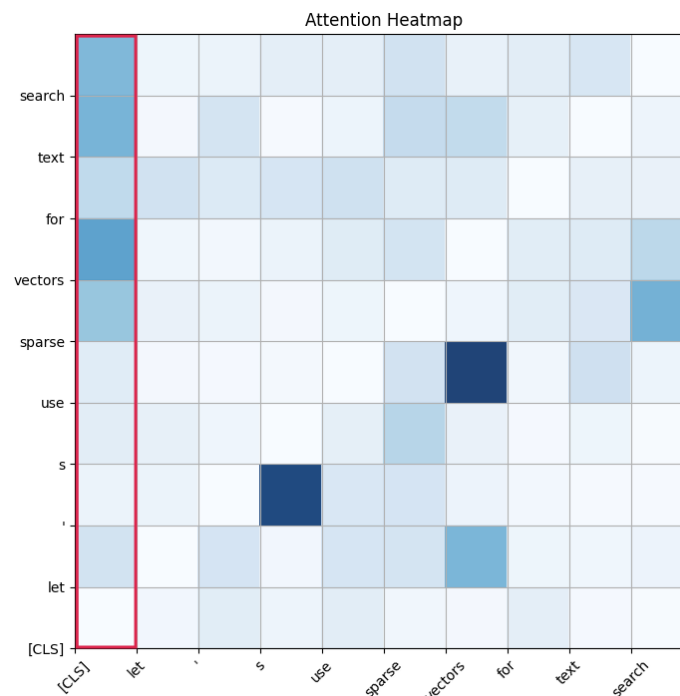


Shorter documents: chunk in RAG

- Hạn chế về độ dài tài liệu: BM25 hoạt động tốt với các tài liệu dài như trang web hoặc sách, nhưng trong RAG, tài liệu được chia thành các đoạn nhỏ cố định.
- Giảm tầm quan trọng của từ khóa: Tần suất xuất hiện từ (term frequency) không còn hiệu quả trong các đoạn văn ngắn, nơi từ chỉ xuất hiện 0 hoặc 1 lần, làm giảm độ phù hợp của BM25 khi áp dụng cho RAG.

- Phiên bản state-of-the-art so với BM25 – SPLADE
- Các tài liệu được biểu diễn dưới dạng vec tơ thưa, với mỗi token là 1 phần tử của vector thưa.
- Gán tất cả trọng số cho các token, vì vậy chúng ta sẽ không cần phải lo lắng về số liệu thống kê và siêu tham số.
- Nhược điểm:
 - Bộ tách từ không hợp lý
 - Mở rộng token tốn kém
 - Phụ thuộc vào miền và ngôn ngữ
 - Thời gian suy diễn: chậm, yêu cầu GPU để thực hiện suy diễn trong thời gian hợp lý.

- [CLS] token đại diện cho cả câu trong tác vụ phân loại
- Công thức BM42 :



$$\text{score}(D, Q) = \sum_{i=1}^N \text{IDF}(q_i) \times \text{Attention}(\text{CLS}, q_i)$$

- [CLS] token đại diện cho cả câu trong tác vụ phân loại

$$\text{score}(D, Q) = \sum_{i=1}^N \text{IDF}(q_i) \times \text{Attention}(\text{CLS}, q_i)$$

- Công thức BM42 :

Vấn đề tách từ

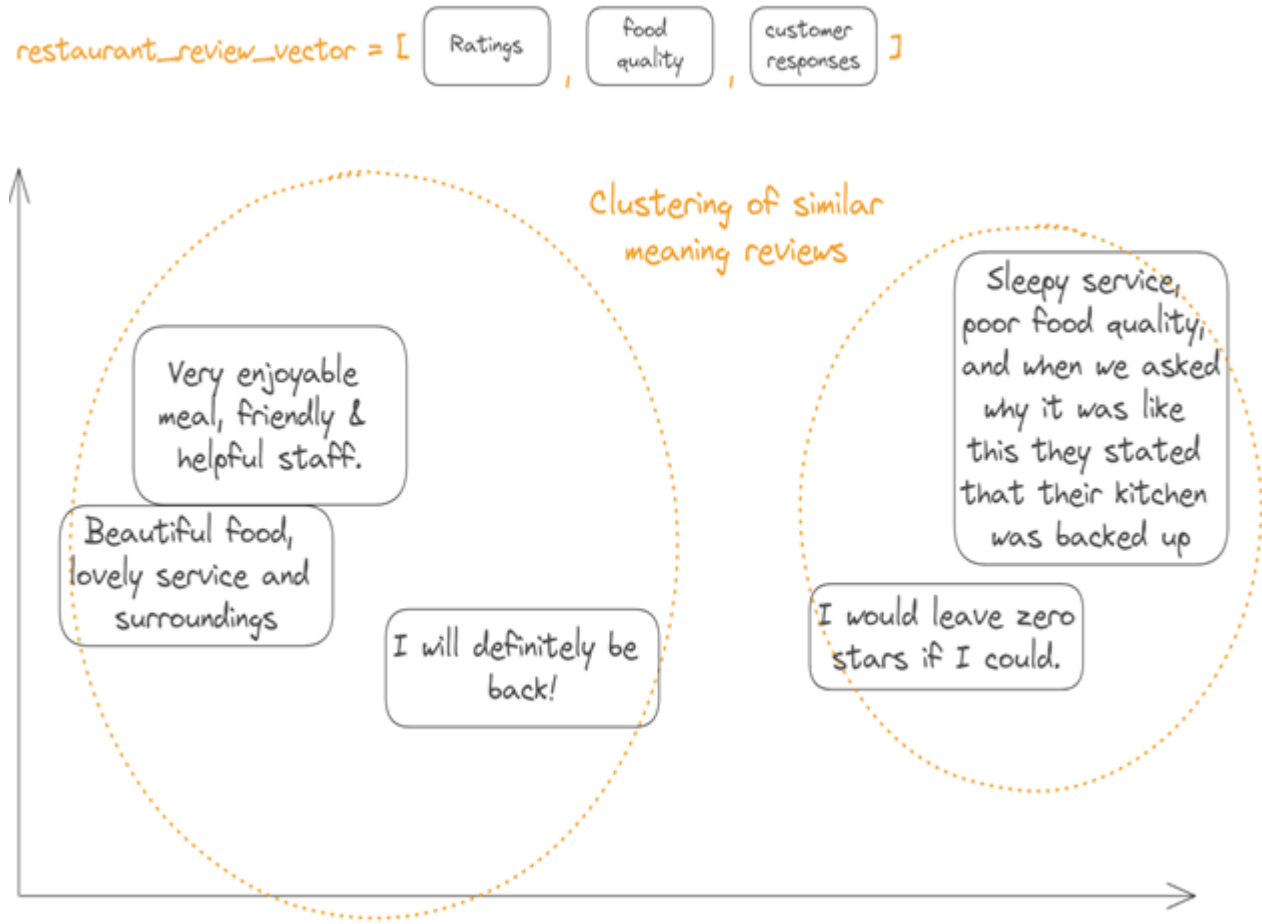
"unbelievable" ->
["un", "##believ",
"##able"]

Chia từ thành subword, sau khi lấy attention thì trộn lại

Dense vector search

- Các từ được nhúng vào vector dày đặc để biểu diễn ngữ nghĩa.
- Tính khoảng cách bằng cách sử dụng độ tương đồng cosine hoặc tích vô hướng.
- Một số framework phổ biến: OpenAI Embedding, Sentence Transformer, Google Embedding.

Embedding process



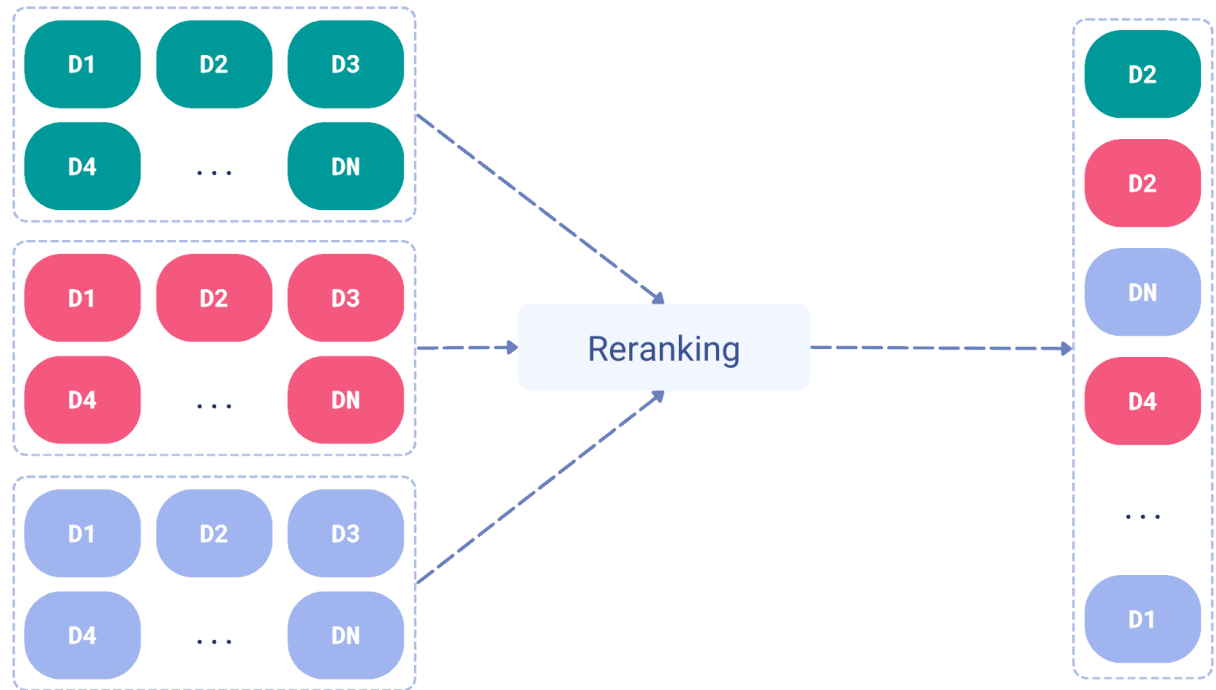
- **IVF (Inverted Index File)**

- IVF chia không gian tìm kiếm thành các vùng khác nhau (cụm) bằng cách áp dụng các phương pháp phân cụm (thường là K-Means) trên tập dữ liệu

- **HSWS (Hierarchical Navigable Small-World Graphs):** thuật toán **Approximate Nearest Neighbor (ANN)** dùng để tìm kiếm các phần tử gần nhất trong không gian vector một cách nhanh chóng và hiệu quả.

Hybrid search - Xếp hạng

Tìm kiếm dày đặc và thưa được thực hiện riêng, sau đó chuẩn hóa điểm số và gộp/rerank để liệt kê kết quả.



$$\text{final_score} = \alpha \times \text{dense_score} + (1 - \alpha) \times \text{sparse_score}$$

Hybrid search – Gộp

- Điểm của tìm kiếm dày đặc và thưa được chuẩn hóa và kết hợp để có được điểm số cuối cùng, sau đó được sử dụng trong việc xếp hạng.

